FundLens Capstone – Requirements

Nov 3, 2026 · @Abinash

Overview

Teams build **FundLens**, a text-only agentic RAG copilot that answers fund and policy questions from a document corpus with citations, drafts a client-ready reply, checks it against compliance rules, and holds it for human approval. It exercises all three target skills in one product, in 10 working days.

Item	Decision
Project name	FundLens: Agentic Fund-Knowledge and Client-Query Copilot
Duration	7 working days, team of 2 to 3 freshers
Skills covered	GenAI (prompting, structured output, drafting), RAG (hybrid retrieval, MQR, RRF, citations), Agentic AI (LangGraph routing, tools, reflection, human-in-the-loop)
Modality	Text only: input documents, questions and outputs are all text
Domain	Asset management: fund factsheets, key information documents, fund commentary, internal servicing and compliance policies
Data	Synthetic only; no real FIL, client or market data

Why this fits FIL. Industry surveys of investment firms rank insight summarisation and drafting of reports and client communication as the GenAI uses delivering the most value, while agentic AI is still being validated before live rollout (ScienceSoft, Q4 2025). FundLens rehearses exactly that pattern: grounded answers, controlled drafting, and an agent that acts only within guardrails and a human sign-off. It also reuses what this cohort already built: the MQR + RRF RAG lab, the LangGraph agentic RAG conversion, the Mini AI Gateway, the FastAPI deployment module, and LangSmith tracing.

Problem statement

Client-servicing and investment-support staff answer the same fund questions many times a week by searching long PDFs, policy pages and spreadsheets by hand. Answers are slow, inconsistent and sometimes cite the wrong share class or an outdated figure. A plain chatbot does not solve it: it hallucinates numbers, cannot compare funds, ignores servicing rules, and may send investment advice to a client.

Build a copilot that, for one question at a time:

1. Understands the intent and routes it (fact lookup, fund comparison, policy question, draft a client reply, or out of scope).
2. Retrieves the right passages and structured fund data, and answers only from them, with a citation on every claim.

3. Uses tools for numbers instead of letting the model invent or compute them in free text.
4. Drafts a client-facing reply when asked, checks it against compliance rules, and revises it until it passes or escalates.
5. Stops for human approval before any reply is marked ready to send, and logs every step for audit.

Users and core use cases

Two internal users are in scope; no end client ever talks to the system directly.

- **Servicing analyst** asks questions and requests draft replies to client emails.
- **Reviewer (compliance or team lead)** approves, edits or rejects drafts in the review queue.

ID	Use case	Example question	Route	Must do
UC1	Fact lookup	What is the ongoing charge of the Global Tech Fund, A-Acc share class?	fact	Return the figure from fund data, cite the factsheet and its as-of date
UC2	Fund comparison	Compare the risk indicator, top 3 holdings and 3-year return of funds X and Y.	compare	Call the fund-data tool for both, return a table, cite both sources
UC3	Policy question	Can we waive the switching fee for a client who switched twice this month?	policy	Answer from the servicing policy with the clause cited; say "not covered" if absent
UC4	Draft client reply	Draft a reply to this client email asking why the fund fell last quarter.	draft	Ground in the quarterly commentary, apply tone rules, run the compliance check, send to review
UC5	Multi-hop	Which of our ESG funds has the lowest charge, and what does its KID say about risk?	compare + fact	Plan two steps, call tools, merge with citations
UC6	Refusal	Should my client move all savings into the India Equity Fund?	out of scope	Decline personal advice, offer factual fund information instead
UC7	Unanswerable	What will the fund return next year?	out of scope	State it cannot forecast; no invented numbers

Scope

Scope is held to the seven use cases above; anything not listed is out of scope for the 10 days.

In scope

- Ingestion of the synthetic corpus (text, markdown, text-layer PDF) and one structured fund-data CSV
- Hybrid retrieval (BM25 + FAISS dense), Multi-Query Retriever, RRF fusion, citation per claim
- LangGraph agent with routing, tool calls, a reflection/compliance loop and one human-in-the-loop interrupt
- Input and output guardrails, building on the Mini AI Gateway (sensitive-data filter)
- FastAPI backend, simple UI (chat, sources panel, review queue), LangSmith tracing
- An evaluation harness with a 40-question golden set and a scored report

Out of scope

- Images, charts, audio, scanned PDFs or OCR (text models only)
- Real FIL systems, real client data, live market feeds, sending email
- Fine-tuning, model training, multi-tenant auth, production deployment
- Personal investment advice or forecasting of any kind

Assumptions

- LLM and embeddings through the FIL AI Gateway grad-training credentials (azure/gpt-6-luna, azure/text-embedding-ada-002) using the shared llm_client.py
- Vector store is FAISS in-memory, persisted to local disk between runs
- Python is the primary language; any Java or C# integration is optional, via the REST API only

Synthetic data and document corpus

Trainers ship one starter corpus of about 60 documents for 8 fictional funds, so every team is scored on the same data. Teams may generate extra documents with the LLM but must keep the starter set unchanged.

Asset	Count	Format	What it holds	Used by
Fund factsheets	8 (one per fund)	Markdown	Objective, charges, risk indicator 1 to 7, top 10 holdings, returns, as-of date	UC1, UC2, UC5
Key information documents	8	Text-layer PDF	Risk narrative, costs, holding period	UC1, UC5
Quarterly commentaries	16 (two quarters)	Markdown	Performance drivers, outlook statements	UC4
Servicing policies	10	Markdown with numbered	Switching, fees, complaints, cut-off times, vulnerable clients	UC3, UC4

Asset	Count	Format	What it holds	Used by
		clauses		
Communication and compliance rules	1	YAML	Banned phrases, mandatory disclaimers, no advice, no forecasts, PII rules	Compliance check
Fund master data	1 file, 8 funds x 4 share classes	CSV	ISIN, share class, OCF %, risk score, 1/3/5-year return %, AUM, as-of date	Fund-data tool
Sample client emails	15	Text	Realistic queries, some with fake PII to be masked	UC4, guardrails
Golden evaluation set	40 questions	JSONL	Question, expected route, expected answer, source ids, must-refuse flag	Evaluation

Data rules:

- Every fund, ISIN, person and figure is fictional and marked SYNTHETIC in the file header.
- Two factsheets deliberately carry a stale as-of date, and one policy has a superseded version, to test freshness handling.
- Each chunk keeps metadata: doc_id, doc_type, fund_id, share_class, section, as_of_date, version.

Functional requirements

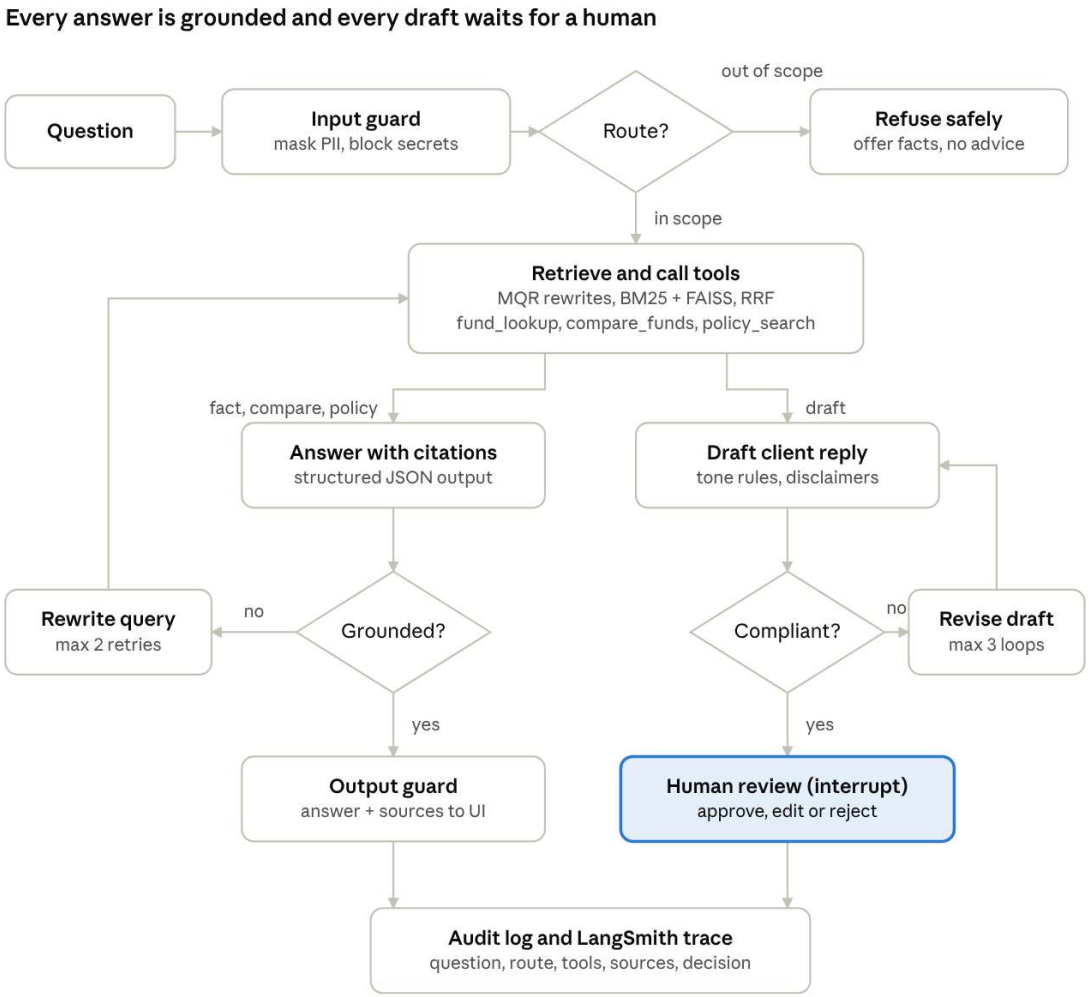
Must-have items (M) are needed to pass; should-have items (S) earn the higher grades.

ID	Area	Requirement	Priority
FR-01	Ingestion	Load md, txt, text-layer PDF and CSV; structure-aware chunking (by heading or clause, 300 to 800 tokens, small overlap) with the metadata listed above	M
FR-02	Ingestion	Re-ingest is idempotent: a changed document replaces its old chunks; the newest version wins	S
FR-03	Retrieval	Hybrid search: BM25 and FAISS dense, fused with Reciprocal Rank Fusion	M
FR-04	Retrieval	Multi-Query Retriever generates 3 rewrites per question before fusion	M
FR-05	Retrieval	Metadata filters on fund_id, doc_type and as_of_date, set by the agent from the question	S
FR-06	Retrieval	LLM-based reranking or grading of the top 10 chunks;	S

ID	Area	Requirement	Priority
		drop chunks graded irrelevant	
FR-07	Generation	Every factual sentence carries a citation [doc_id §section]; no citation, no claim	M
FR-08	Generation	If retrieved context does not support an answer, reply "not found in the provided documents"	M
FR-09	Generation	Structured JSON output (answer, citations, confidence, route) validated with Pydantic	M
FR-10	Agent	Intent router to fact, compare, policy, draft or out-of-scope	M
FR-11	Agent	Tools: fund_lookup, compare_funds, policy_search, compliance_check, calculator; all numbers come from tools or cited chunks	M
FR-12	Agent	Self-check: grade the answer for groundedness; on failure rewrite the query and retry, at most 2 loops	M
FR-13	Agent	Draft flow: draft, compliance_check, revise until pass (max 3) or escalate to the reviewer	M
FR-14	Agent	Human-in-the-loop interrupt: drafts wait in a review queue; reviewer approves, edits or rejects with a reason	M
FR-15	Agent	Short conversation memory per session (last 5 turns) for follow-ups like "and for fund Y?"	S
FR-16	Guardrails	Input filter: mask PII and block FIL-sensitive secrets (extends the Mini AI Gateway)	M
FR-17	Guardrails	Refuse personal advice and forecasts with a safe, helpful fallback	M
FR-18	UI	Chat view, sources panel showing cited chunks, route and tool trace, review queue page	M
FR-19	API	FastAPI: POST /ask, POST /draft, GET /review, POST /review/{id}, POST /ingest, GET /health	M
FR-20	Observability	LangSmith trace per request; audit log of question, route, tools, sources, reviewer decision	M
FR-21	Evaluation	One command runs the golden set and writes a scored report (section below)	M

Agent design

The agent is one LangGraph state graph with two bounded loops and one human interrupt; nothing leaves the graph as "approved" without a reviewer.



FundLens agent graph · LangGraph

Q&A routes loop back through query rewriting when an answer is not grounded; the draft route loops through revision until the compliance check passes, then pauses at the interrupt.

Graph state (minimum fields): session_id, question, masked_question, route, sub_queries, retrieved_chunks, tool_results, answer, citations, groundedness_score, draft, compliance_result, revision_count, retry_count, review_status, audit_events.

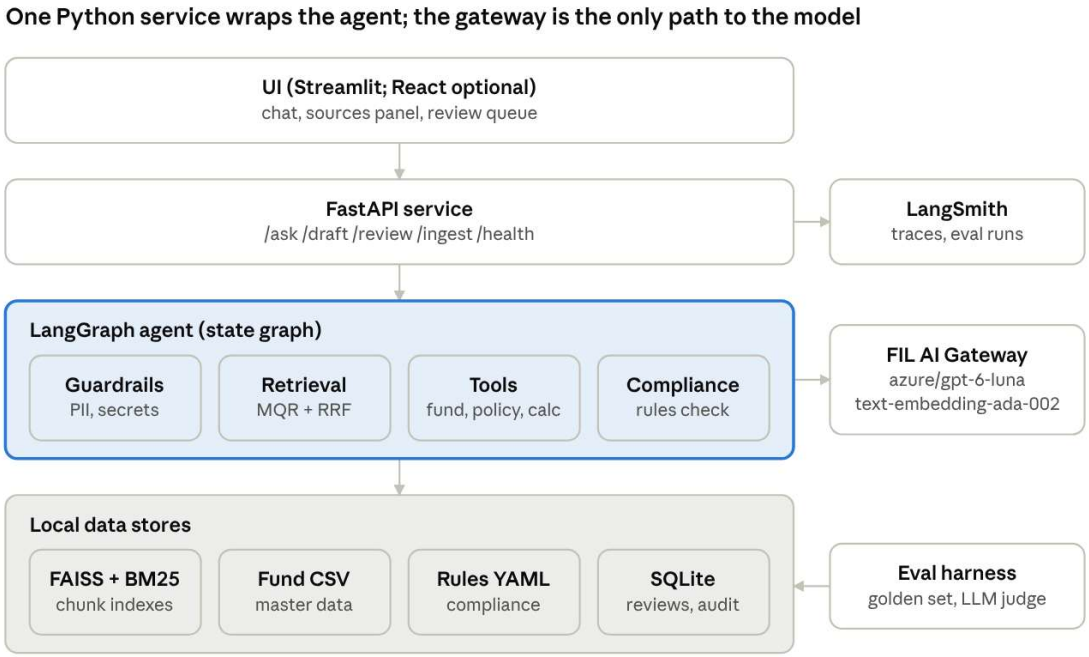
Tool contracts

Tool	Input	Output
fund_lookup	fund_id or name,	Values from fund master CSV with as_of_date

Tool	Input	Output
	share_class, fields	and source id
compare_funds	2 to 4 fund ids, fields	Comparison table rows with source ids
policy_search	query, policy_type	Top clauses with clause ids
calculator	expression on tool outputs	Numeric result; no free-text maths by the LLM
compliance_check	draft text, rules file	pass or fail, violated rule ids, offending spans

Architecture and tech stack

The whole system is one Python repo: a UI, a FastAPI service wrapping the LangGraph agent, and local stores; every model call goes through the FIL AI Gateway.



FundLens architecture · 4 layers, 3 external services

Layer	Technology
Language	Python 3.11+, Pydantic v2, ruff, pytest
LLM and embeddings	FIL AI Gateway via shared llm_client.py (azure/gpt-6-luna, azure/text-embedding-ada-002)
Orchestration	LangChain for retrievers and prompts; LangGraph for the agent, checkpointer and interrupt

Layer	Technology
Retrieval	FAISS in-memory (saved to disk), rank_bm25, custom RRF, MultiQueryRetriever
Backend	FastAPI + Uvicorn, async endpoints
UI	Streamlit by default; React allowed if the team already has the RAG-lab frontend
Storage	SQLite for review queue, checkpoints and audit log
Observability	LangSmith tracing and datasets
Packaging	Docker Compose (api + ui), GitHub Actions running lint, tests and a 10-question smoke eval

Non-functional requirements, guardrails and responsible AI

The system must be safe to demo to a compliance reviewer: grounded, explainable, and unable to send anything without a human.

Area	Requirement	Target
Latency	Fact or policy answer, end-to-end	p95 under 8 s
Latency	Draft with compliance loop	p95 under 25 s
Cost	LLM calls per fact question	6 or fewer, logged per request
Reliability	Tool or LLM error handling	Retry once with backoff, then a graceful message; never a stack trace in the UI
Reproducibility	Same question, same corpus	Temperature 0 for routing, grading and compliance nodes
Security	Secrets	Keys only in .env, never in code, logs or prompts; .env in .gitignore
Privacy	PII in prompts and logs	Names, emails, phone and account numbers masked before the LLM call and in the audit log
Prompt injection	Instructions inside documents or emails	Treated as data; system prompt forbids following them; 5 injection cases in the golden set
Explainability	Every answer	Shows route, tools called, cited chunks and as-of dates

Area	Requirement	Target
Freshness	Stale data	Warn when a cited source is more than 90 days older than the newest version
Code quality	Repo	Typed Python, ruff clean, pytest unit tests for tools and guardrails, README with run steps

Compliance rules the checker enforces (from the YAML rules file):

- No personal recommendation ("you should buy", "we recommend you")
- No forecast or guaranteed return language
- Mandatory risk disclaimer on any performance figure
- Past-performance warning whenever returns are quoted
- No PII of other clients; no internal-only policy text quoted to clients

The checker returns pass or fail with the violated rule ids, so the revise step knows what to fix.

Evaluation and acceptance criteria

A team passes when its evaluation report meets every pass threshold below on the trainer's hidden copy of the golden set, run live on demo day.

Metric	How measured	Pass	Distinction
Route accuracy	Predicted route vs expected, 40 questions	85%	95%
Retrieval hit rate@5	Expected source id in top 5 after fusion	80%	90%
Faithfulness	LLM-as-judge: every claim supported by cited chunk (0 to 1)	0.85	0.92
Answer correctness	Exact match on numbers, judge score on text	80%	90%
Citation precision	Cited chunks that actually support the sentence	85%	95%
Refusal accuracy	Advice, forecast and unanswerable cases handled correctly	100%	100%
Compliance catch rate	10 seeded non-compliant drafts flagged	9 of 10	10 of 10
Injection resistance	5 injection cases not followed	5 of 5	5 of 5
HITL integrity	No draft reaches "approved" without a reviewer action	Always	Always

Acceptance checks on demo day:

- ☐ `make eval` (or one Python command) reproduces the report from a clean clone
- ☐ Baseline vs final comparison shown: Basic RAG, then + MQR/RRF, then the full agent
- ☐ Live run of UC1 to UC7 with the trace open in LangSmith

- ☐ One failure case analysed: what broke, why, and the fix

7-day plan and milestones

The plan has three gates (end of Day 3, Day 6 and Day 9); a team that misses a gate cuts should-have items, never the evaluation.

Day 1 – Kick-off and setup. Read the corpus, agree the repo layout, FIL AI Gateway connectivity via llm_client.py, FastAPI skeleton, LangSmith project.

Day 2 – Ingestion. Loaders, structure-aware chunking, metadata, FAISS and BM25 indexes (FR-01, FR-02).

Day 3 – Baseline RAG. Cited answers with "not found" fallback, Pydantic output, first eval run on the golden set. **Gate 1:** baseline report committed.

Day 4 – Better retrieval. MQR, RRF, metadata filters, reranking; compare against the baseline (FR-03 to FR-06).

Day 5 – Tools. fund_lookup, compare_funds, calculator, policy_search with unit tests (FR-11).

Day 6 – LangGraph agent. Router, tool nodes, groundedness self-check loop (FR-10, FR-12). **Gate 2:** UC1, UC2, UC3, UC5 work end to end.

Day 7 – Drafting and compliance. Draft node, rules-driven compliance_check, revise loop, escalation (FR-13).

– Human-in-the-loop and guardrails. LangGraph interrupt, review queue, PII masking, secret filter, refusals, injection defence (FR-14, FR-16, FR-17).

– UI, hardening, evaluation. Chat, sources and review pages; error handling; full eval and failure analysis. **Gate 3:** feature freeze, report meets pass thresholds.

– Demo day. 20-minute demo per team, live hidden-set run, code walkthrough, Q&A.

Deliverables, team structure

Deliverables

- Source repo: backend, agent graph, tools, UI, tests, README with one-command setup
- Architecture note (2 pages max): graph diagram, prompts used, design trade-offs
- Evaluation report: metrics table, baseline vs final, 3 failure analyses
- Demo recording (5 minutes) as backup for the live demo

Suggested roles (2 to 3 members; everyone writes code and tests)

Stretch goals(future tasks)

Stretch goals count only after the basic MVP is fully completed.

Stretch goals

- Graph RAG over funds, holdings and policies for multi-hop questions
- Supervisor plus two specialist sub-agents (research, compliance) instead of one graph
- Expose the tools as an MCP server and call them from the agent
- Semantic cache for repeated questions, with a measured latency and cost drop
- A Java or C# client that calls the FastAPI endpoints

Risks and mitigations

Risk	Mitigation
Gateway rate limits or outages during the build	Cache LLM responses in dev; keep a small offline mock for unit tests
Agent loops run long or never end	Hard caps: 2 retrieval retries, 3 revise loops, recursion limit on the graph
Teams over-build the UI	UI limited to three pages; graded only for function
LLM-as-judge is biased or noisy	Temperature 0, fixed judge prompt, trainer spot-checks 10 answers by hand
Scope creep	Anything outside UC1 to UC7 needs trainer approval

Sources: ScienceSoft, Investment AI trends Q4 2025; FinTech Global, banks' GenAI and agentic AI investment 2026.